TellaVault

SAVE · INVEST · GROW

PROTOTYPE BUILD PLAN & PROOF OF CONCEPT

A savings & investment platform, web, installable mobile app & admin console

A working proof of concept demonstrating end-to-end product capability: onboarding, wallet, savings, investments, role-based admin, live activity logs and in-app support, built to show the client we can ship the real thing.

PREPARED BY

Swizel Technologies Limited

You Imagine, We Build · swizel.co

PREPARED FOR

Mr Joshua Edobor

Draft v2 · 1 June 2026

01 · OVERVIEW

What we're building

A working prototype of **TellaVault**, a savings and investment platform in the spirit of Cowrywise, built to prove to the client that Swizel can deliver the real product. It is a **proof of concept**: every flow works end to end, but money movement is **simulated**, so we demonstrate the full experience without payment licensing, KYC vendors or a live brokerage.

The prototype ships as three connected surfaces on one backend:

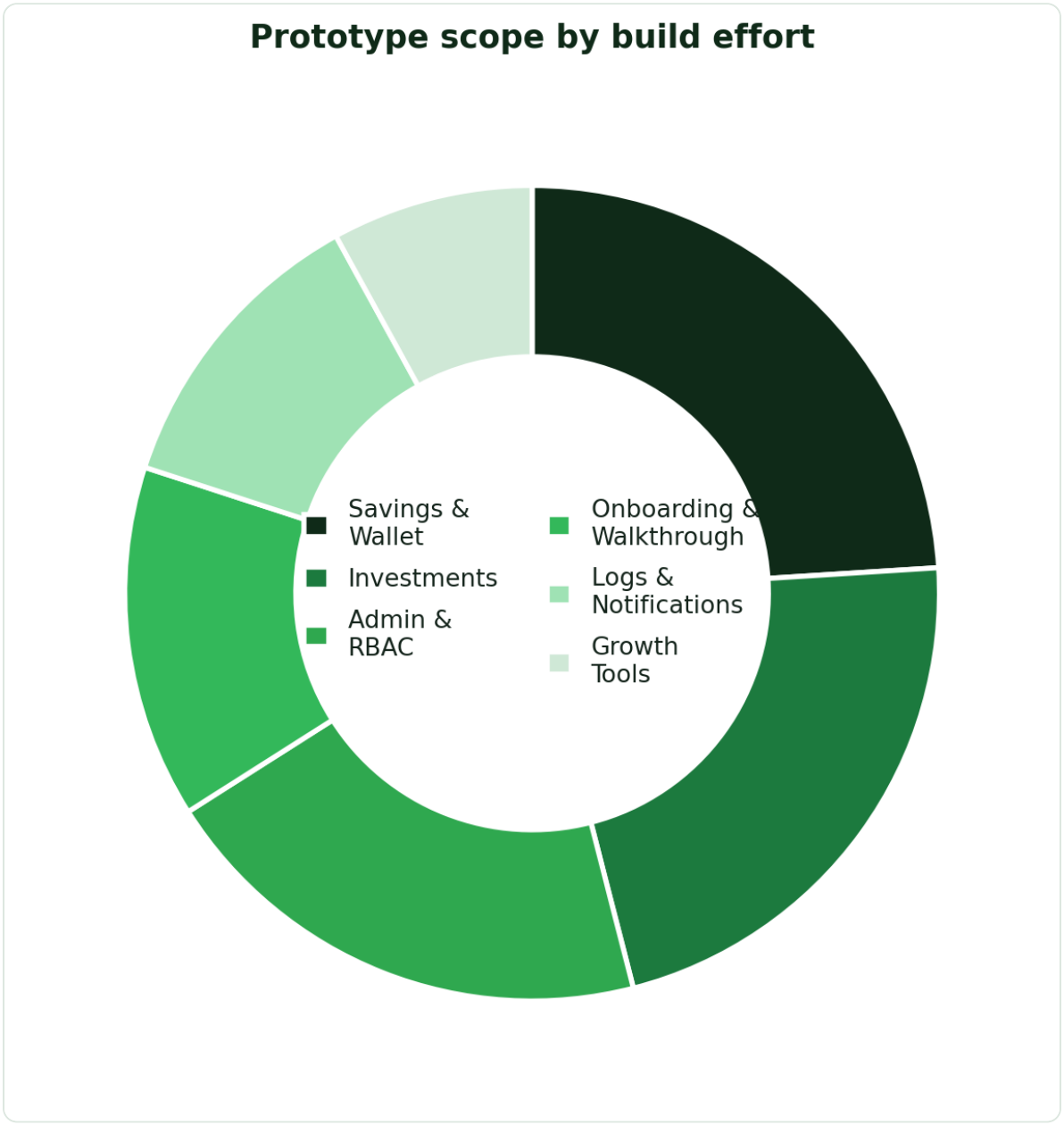
1 · Web app
The customer-facing product: onboarding, wallet, savings, investments and growth tools, with a guided first-time walkthrough.

2 · Mobile app (PWA)
The *same* app delivered as an installable, responsive Progressive Web App with a full splash-screen flow, "add to home screen" for a true app feel.

3 · Admin console
A separate dashboard with role-based access control for operations, finance, support and compliance, including the live activity-logs page.

Confirmed decisions

Area	Decision
Product name & brand	**TellaVault**, forest-green & black identity, diagonal-stripe mark
Mobile delivery	Installable **PWA** with full splash-screen workflow (single codebase, Netlify-hosted)
Backend	**Firebase**, Auth, Firestore, Cloud Functions, Storage
Feature scope	Savings + wallet · Investments · Admin + RBAC · Growth tools · Onboarding walkthrough · Notifications · Live support chat · Activity logs
Fidelity	**Working flows with simulated money** (real auth + DB + CRUD; no live gateway)
Dev workflow	**VS Code** + **GitHub** repo + **Netlify** continuous deploys with per-branch preview URLs

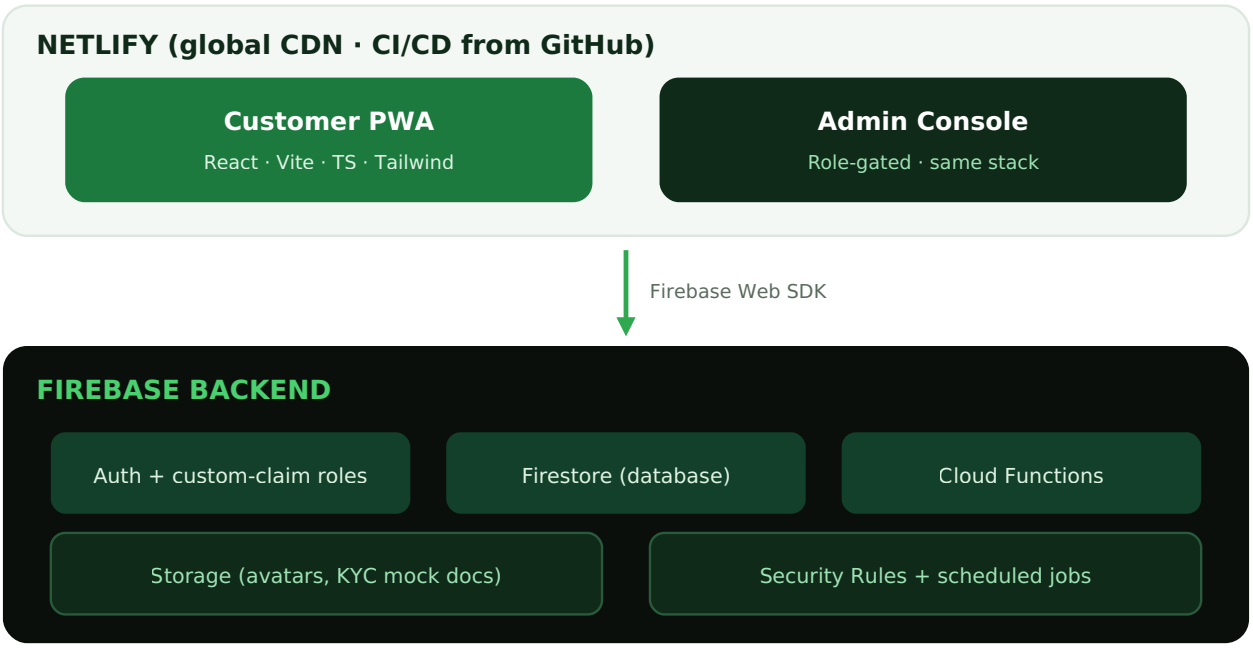


Relative build effort across the prototype's feature areas.

02 · ARCHITECTURE

System architecture

A single React + Vite + TypeScript codebase (styled with Tailwind) powers the customer web app and the installable mobile PWA. A separate admin app shares the stack. Both deploy to Netlify; both talk to one Firebase backend.



Why this stack

One React codebase gives the polished, responsive UI the product needs while staying fast to build. Firebase removes server management and provides auth, a real-time database and clean RBAC through custom claims. Netlify gives the client a shareable preview URL on every push. It is all something a small team ships quickly and a larger team later hardens into production.

Customer & admin (front-end)

- React 18
- Vite
- TypeScript
- Tailwind
- React Router
- TanStack Query
- vite-plugin-pwa
- Recharts
- Zod + RHF

Backend

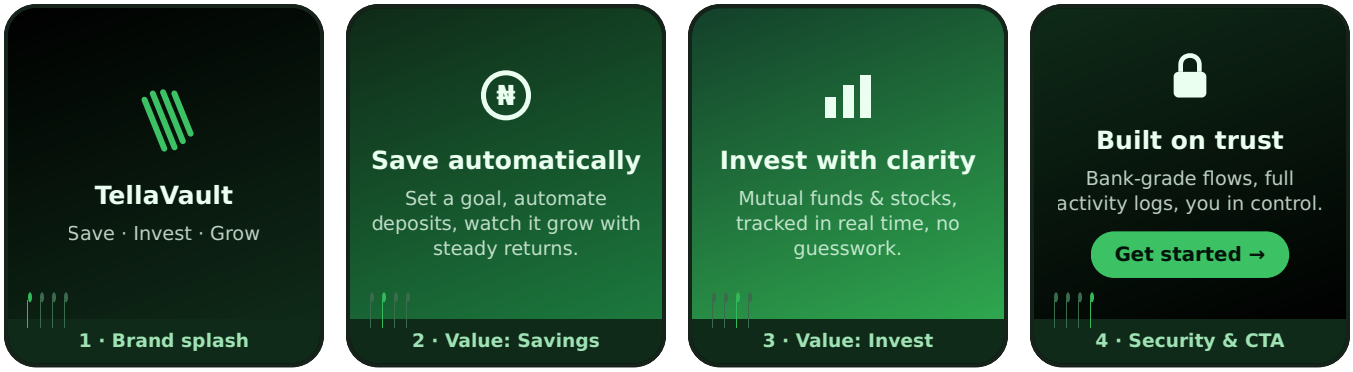
- Firebase Auth
- Firestore
- Cloud Functions (TS)
- Storage
- Security Rules
- Scheduled jobs

03 · EXPERIENCE

Splash workflow & first-time walkthrough NEW

First impressions matter. New users meet TellaVault through a sequenced **splash-screen workflow** on the PWA, then an interactive **step-by-step walkthrough** on the web that any first-time user can click through, using short looping videos, animated SVGs and tooltips to show exactly how the product works before they commit.

A · Splash-screen sequence (mobile PWA launch)



Splash sequence shows once on first launch, is skippable, and is re-accessible from Settings. A small **"Demo"** badge stays visible so balances are never mistaken for real funds.

B · Interactive web walkthrough (first-time users)

On the website, a "**See how it works**" launcher opens a guided, clickable tour. Each step pairs a short looping video / animated SVG with a highlighted UI hotspot and a tooltip:

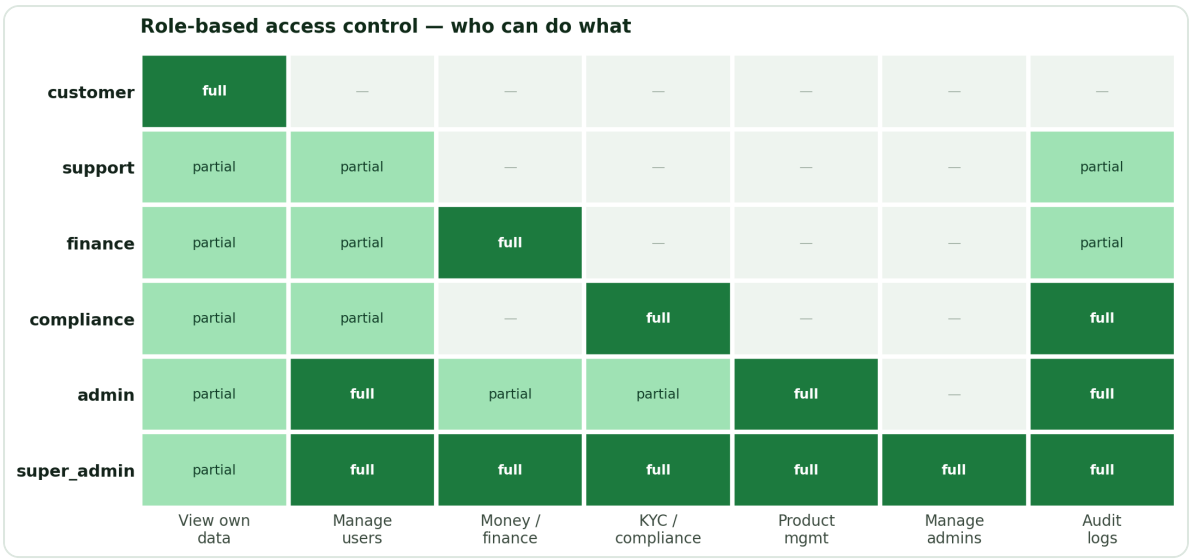
1. **Create your account**, email sign-up with a progress meter; animated SVG shows the 3-step flow.
2. **Fund your wallet**, looping clip of the simulated funding flow crediting the wallet instantly.
3. **Create a savings plan**, tooltip tour of choosing a plan type, target and frequency.
4. **Make an investment**, walk through picking a fund/stock and placing a buy order.
5. **Track & get notified**, see the dashboard charts and the notifications centre update live.

The walkthrough is built as a reusable **tour engine** (steps defined in config), so the client can see new features can be onboarded later without re-engineering, a strong signal of a production-minded build.

04 · ACCESS CONTROL

Roles & RBAC

Access is governed by a **role** stored as a Firebase custom claim and mirrored in Firestore for queryability. Cloud Functions and Security Rules both check the claim, so a user can never elevate themselves from the client. Every privileged action writes to an immutable audit trail.



Six roles across seven capability groups. "Full" = write/manage, "partial" = scoped read or limited action.

Role	Surface	Primary responsibilities
customer	Web / PWA	Own account only: wallet, savings, investments, growth tools
support	Admin	Read users, plans, transactions; handle tickets & chat, no money actions
finance	Admin	View & reconcile transactions, approve simulated payouts, run reports
compliance	Admin	Review KYC, freeze/flag accounts, read full audit logs
admin	Admin	Manage products, users and most operations
super_admin	Admin	Everything, including managing other admins' roles

05 · DATA MODEL

Database design (Firestore)

Modeled relationally for clarity even though Firestore is document-based. Every balance change is a **transaction** record, a single source of truth for all money movement, and every privileged action is an append-only **audit log**.

users

- **uid** (PK)
- name, email, phone
- role, status
- kycStatus, riskProfile

wallets

- **uid** → users
- availableBalance
- lockedBalance
- currency (NGN/USD)

transactions

- **txnId** (PK)
- userId → users
- type, amount, status
- balanceAfter, reference

savings_plan_types

- **typeId** (PK)
- name (Regular/Fixed/Goal)
- interestRatePA
- tenor, penalty

savings_plans

- **planId** (PK)
- userId, typeId →
- target, current, frequency
- maturityDate, status

interest_accruals

- **id** (PK)
- planId → savings_plans
- amount, accruedAt
- *(scheduled job)*

investment_products

- **productId** (PK)
- assetClass (fund/stock)
- name, ticker, unitPrice
- historicalReturns[]

holdings

- **id** (PK)
- userId, productId →
- units, avgBuyPrice
- currentValue

orders

- **orderId** (PK)
- userId, productId →
- side (buy/sell)
- units, price, status

kyc

- **uid** → users
- idType, docUrl →Storage
- reviewedBy → admins
- status

notifications

- **id** (PK)
- userId → users
- title, body, type
- read, createdAt

support_threads

- **id** (PK)
- userId, agentId →
- messages[], status
- updatedAt

tool_results

- **id** (PK)
- userId (nullable)
- tool, inputs, output
- feeds riskProfile

admins

- **uid** (PK)
- role, permissions[]
- invitedBy → admins
- lastLogin

audit_logs

- **id** (PK) · append-only
- actorId, actorRole
- action, target
- before, after, ip, ts

A seed dataset ships with the build, demo users across every role, sample plans, funds and stocks with believable Nigerian-market figures, so the product is populated and alive on day one of the demo.

06 · ENGAGEMENT & OVERSIGHT

Notifications, live support & activity logs NEW

Notifications centre

A fully working notifications page (web + PWA), driven by the `notifications` collection and updating in real time via Firestore listeners. Items are typed and grouped, with unread badges, mark-as-read, filters by category and an empty/loading/error state for each.

- **Transactions**
- **Savings milestones**
- **Investment updates**
- **Security alerts**
- **Announcements**
- **Support replies**

Live support chat

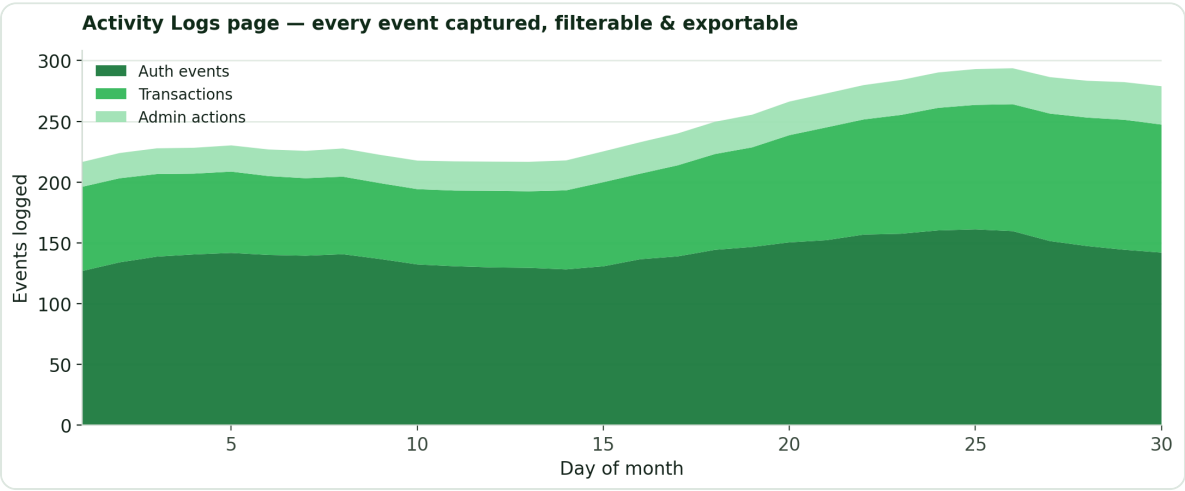
An in-app chat widget connects customers to the admin console in real time. Messages persist in `support_threads`; the **support** role sees an inbox of conversations, can reply live, and resolve threads. Typing indicators, unread counts and a canned-replies panel round it out.

Customer side
Floating chat bubble → live conversation, history preserved, replies arrive as notifications.

Agent side (admin)
Queue of open threads, customer context panel, live reply, assign & resolve, searchable history.

Activity logs, filterable & exportable

A full logs page records **every activity on the platform**, logins, funding, plan changes, orders, admin actions, role changes, each fully detailed (actor, role, action, target, before/after, timestamp, IP). The page is **filterable** (by user, role, action type, date range, severity), **exportable** to CSV / PDF, and renders **charts** (volume over time, by category) with optional **maps** of activity by location.



Sample of the logs analytics view, stacked event volume by category, one of several built-in chart modes.

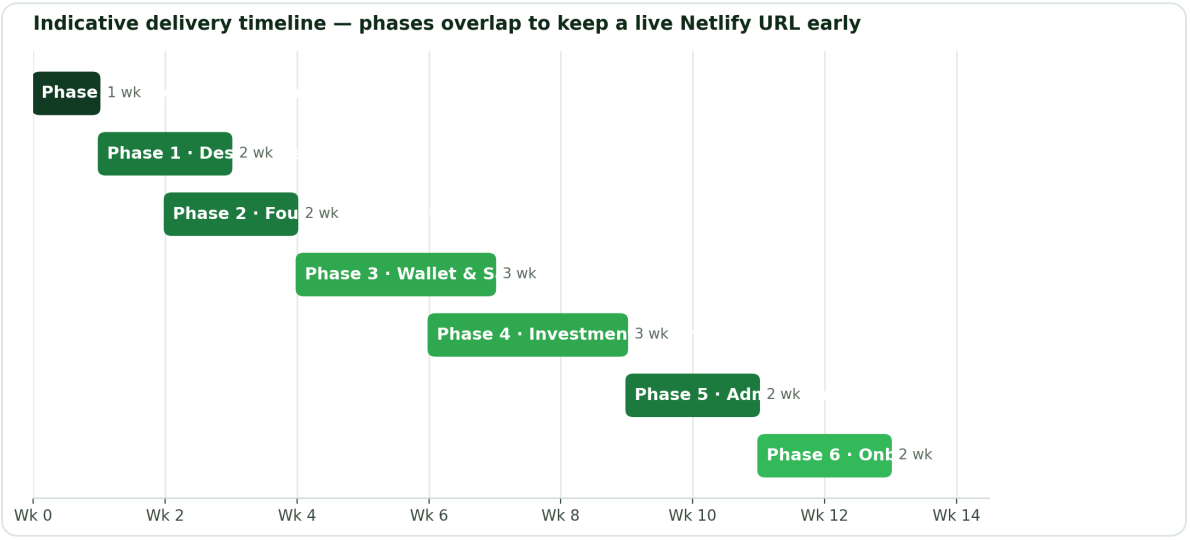
Capability	Detail

Filters	User · role · action type · date range · severity · free-text search
Export	CSV and PDF of the current filtered view
Charts	Events over time, by category, top actors, success vs. failure
Maps	On request, activity plotted by approximate location (IP-derived, demo data)
Integrity	Append-only; admin reads are themselves logged

07 · DELIVERY

Delivery phases

We follow Swizel's own method, **Discover → Design → Build → Ship**, compressed for a prototype. Each phase ends with something the client can see on a live Netlify URL.



Indicative timeline. Sequence is fixed; calendar dates are set at kickoff based on team size.

Phase	Outcome the client sees
0 · Discovery	This plan approved; feature list & demo script locked
1 · Design system & flows	TellaVault visual system + clickable wireframes for key journeys
2 · Foundation	Live skeleton on Netlify, sign up & log in; DB, rules & seed deployed
3 · Wallet & savings	A customer can fund a wallet and save end to end; dashboards live
4 · Investments & growth tools	Funds & stocks, portfolio, and the three growth calculators
5 · Admin & RBAC	Role-gated admin: users, KYC, products, reports, live support inbox
6 · Onboarding, logs & polish	Splash + walkthrough, notifications, activity logs, PWA polish, demo-ready

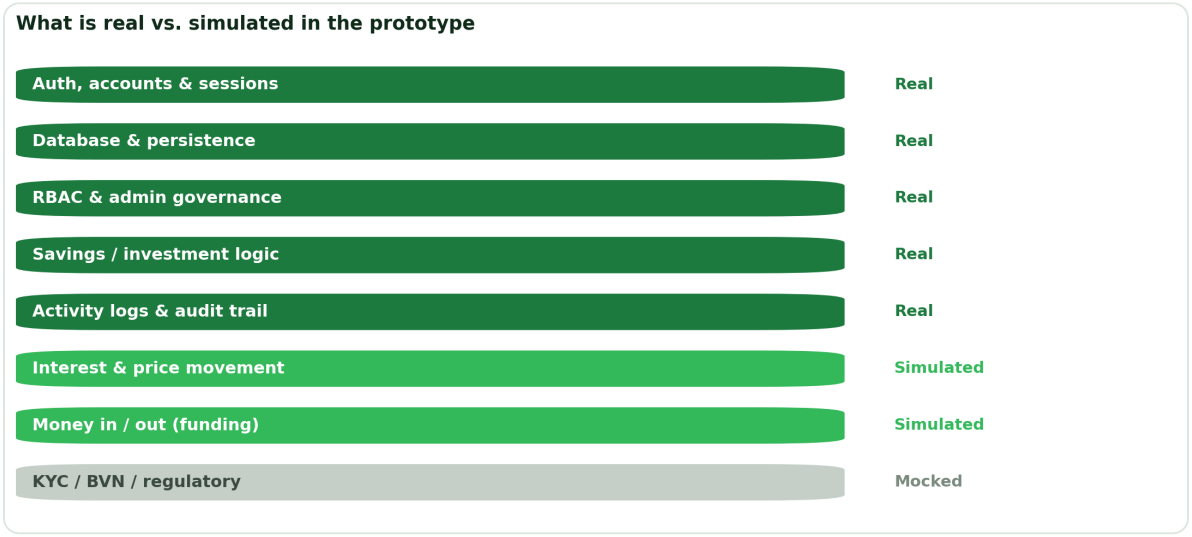
Dev workflow, VS Code · GitHub · Netlify

1. **You build in VS Code** in the folder you've created; we scaffold the monorepo there.
2. **Linked to your GitHub repo**, feature branches, pull requests, conventional commits.
3. **Netlify auto-deploys** every push: production from `main`, a preview URL per pull request for the client to review.
4. **Firebase** runs as separate dev and demo projects; secrets stay in environment variables, never in the repo.

08 · EXPECTATIONS

What's real vs. simulated

So expectations are crystal clear with the client, here is exactly what is genuinely functional versus simulated or mocked in the proof of concept.



Auth, data, governance and all business logic are real; money rails and regulatory checks are simulated.

If the client wants any one of these made "real", for example a **Paystack test-mode** integration so funding feels live, that is a small, well-scoped add-on we flag during the demo. It's a natural upsell into the production build.

Risks & notes

Note	Why it matters
Netlify ≠ app store	The "mobile app" is an installable PWA, not a Play/App Store listing. Native distribution is a production-phase item; the same codebase extends to React Native.
Not audited / regulated	This is a POC, not pen-tested or compliant. We state this plainly so it isn't treated as launch-ready.
"Demo" badge stays visible	Simulated balances must never be mistaken for real funds.
Scope discipline	The feature list in §1 is the boundary; new asks become tracked add-ons.

Recommended next step

Approve this plan, then we kick off **Phase 1 (design system + wireframes)** in parallel with **Phase 2 (foundation scaffold)** so there's a live Netlify URL early for the client to follow. On your word we scaffold the repo in your VS Code folder, link it to GitHub, stand up the Firebase project, and produce the first TellaVault wireframes.